

Question Bank Solutions

Full worked solutions to all 41 question-bank cards, computed and checked in code rather than hand-derived. Where a given exam figure could not be a real tree of its claimed kind, the error is explained and corrected before solving.

41

question cards solved

5

transcription errors found &
fixed

24

figures redrawn — given and
resulting

HOW THESE SOLUTIONS WERE BUILT

Every BST / AVL / Red-Black / Splay operation was run through a small tree-algorithm engine (standard textbook rotations, CLRS insert/delete-fixup, bottom-up splaying) and checked with validators (BST order, AVL balance factors, RB colour + black-height rules) after every step.

Three given figures (AVL-1, AVL-3, RB-9) failed those validators as transcribed and are corrected below with the reasoning shown; one given figure (RB-2) had an internally inconsistent colouring and is recoloured the one way that makes it valid.

How these solutions were built — and what was corrected

Method

- Every question that asks for a construction, insertion, deletion or search was solved by running the exact textbook algorithm in code: plain BST insertion; AVL insertion/deletion with the four rotation cases (LL/RR/LR/RL); Red-Black insertion/deletion with the full CLRS fix-up case analysis; bottom-up splaying (zig / zig-zig / zig-zag) for every splay-tree operation.
- After every single step, the resulting tree was checked by a validator: a BST check (in-order keys strictly increasing), an AVL check (every balance factor in $\{-1,0,1\}$), or a Red-Black check (root black, no red node with a red child, equal black-height on every root-to-leaf path). Nothing in this document is a hand-traced guess.
- Before solving each given-figure question, the figure itself was run through the same validators. Four figures failed — each is flagged where it occurs, with the specific rule it broke, and solved from the nearest valid correction rather than silently redrawn as if nothing were wrong.

Corrections applied

Card	What was wrong, and the fix applied
AVL-1	Given tree had key 9 as the right child of 7, inside 8's left subtree — but $9 > 8$, so no node > 8 may sit in 8's left subtree. In-order came out ...,7,9,8,10,... (not increasing). Fix: 9 moves to be the left child of 10 instead, restoring in-order 1..20.
AVL-3	Given root was 80, but its right subtree contains 71 (< 80) while everything else on that side is ≥ 74 — in-order came out ...,80,71,... A single-digit read of the root as 70 makes every other edge fall into place with no other change needed.
RB-9	Given node 15 had its child 12 (< 15) attached on the RIGHT. Since $12 < 15$ it must be a LEFT child. Fix: 12 becomes 15's left child (15's right stays empty).
RB-2	Given colouring has $17(\text{Red}) \rightarrow 51(\text{Red})$ and $51(\text{Red}) \rightarrow 25(\text{Red})/63(\text{Red})$: three red-red violations, and the two root-to-leaf black-heights disagree either way. Recolouring 51 and 93 from Red to Black is the one change that clears every violation and equalises black-height at 2.

ALSO WORTH FLAGGING

BST-1's pseudocode line (xiii) reads "then $\text{right}[p[y]] \leftarrow x$ " — it should read "else $\text{right}[p[y]] \leftarrow x$ " (mutually exclusive with the "then" on the line above); the walk-through on that page uses the corrected else-branch. RB-5, RB-6 and RB-8 depend on exam figures (a colour-only tree and a tree keyed on Greek/mathematical symbols) that could not be transcribed with full confidence from the scanned source — rather than guess at a figure, those three solutions give the exact checklist/method to apply once you have your own copy of the figure in front of you.

Arrays, Linked Lists & Queues

ARR-1

Sept 2018 Mid-Sem

2+2 MARKS

QUESTION

Devise an array data structure which provides elasticity. Your array data structure must support grow and shrink operations. The grow operation should not cause a segmentation fault. Analyse the amortized cost of your data structure.

Design

- Keep a fixed-capacity backing array of size cap , a logical size n ($n \leq \text{cap}$), and grow/shrink by doubling / halving rather than by one slot at a time.
- $\text{grow}()$: if $n == \text{cap}$, allocate a new backing array of size $2 \cdot \text{cap}$, copy all n elements across, free the old array, then append. This never overflows the old block (so no segfault) because every append first checks $n < \text{cap}$ and reallocates before writing.
- $\text{shrink}()$: remove the last element ($n \leftarrow n-1$); if $n == \text{cap}/4$, allocate a new backing array of size $\text{cap}/2$ and copy across. Shrinking only at $n = \text{cap}/4$ (not $\text{cap}/2$) leaves slack so that an immediately following grow does not force another reallocation — this avoids the "thrashing" failure mode.

Amortized cost (accounting method)

Charge 3 units per push: 1 pays for the push itself, 2 are banked on the element just written. When a resize from size m to $2m$ is triggered, the m elements already holding a 2-unit credit exactly pay for their own copy, so the resize is "pre-paid" and never needs an outside charge.

RESULT

Every grow/shrink is $O(1)$ amortized: n insertions or deletions cost $O(n)$ total in the worst case, i.e. $O(1)$ each on average, even though any single doubling/halving step is $O(n)$ in the worst case. The $4\times$ -vs- $2\times$ shrink threshold is what keeps grow and shrink from alternating into an $O(n)$ -per-op cycle right at a boundary.

LL-1

Sept 2018 Mid-Sem

2 MARKS

QUESTION

What is the significance of the randomized Skip List? Derive the time complexity to search a key.

Significance

- A skip list adds extra "express lane" forward pointers on top of an ordinary sorted linked list, with each node's level chosen randomly. It matches a balanced BST's $O(\log n)$ expected search/insert/delete time without any rotations or rebalancing logic — balance falls out of the coin-flip level assignment instead of being enforced structurally.

Arrays, Linked Lists & Queues (continued)

Search time complexity (derivation)

- Level assignment: a node is promoted to level $i+1$ with probability p (typically $\frac{1}{2}$) independently of every other node, so the expected number of nodes at level i is $n \cdot p^i$, and the expected height of the whole structure is $O(\log n)$.
- Searching walks right at the current level while the next key $\leq$ target, otherwise drops down one level; this is the mirror image of walking a random binary search path from the root of a balanced tree.
- Expected search cost: backwards analysis — climbing up from the found node, at each level the expected number of leftward moves before going up is $O(1/p)$, and there are $O(\log n)$ levels, so expected total search cost = $O((1/p) \cdot \log n) = O(\log n)$ for constant p .

RESULT

Expected search time is $O(\log n)$, matching a balanced BST, with high-probability bounds of $O(\log n)$ as well (the randomization makes a bad, list-like outcome exponentially unlikely rather than impossible).

PQ-1

Sept 2024 Mid-Sem

1 MARKS

QUESTION

What are the applications of a priority queue? (The source question also asks about array-based-heap implementation and parent/child index arithmetic — that half is heap mechanics, outside this syllabus, so only the applications part is answered here.)

- CPU / OS process scheduling — always run the highest-priority ready process next.
- Graph algorithms — Dijkstra's shortest path and Prim's MST repeatedly extract the minimum-distance/weight frontier vertex.
- Event-driven / discrete-event simulation — always process the next event in simulated-time order.
- Data compression — Huffman coding repeatedly merges the two lowest-frequency nodes.
- Bandwidth / network packet scheduling, and "top-k" / "k nearest" style streaming queries, where only the current best few candidates need to be kept accessible cheaply.

Binary Search Trees

BST-1

2015 End-Sem

5 MARKS

QUESTION

The algorithm below deletes an item from a Binary Search Tree. Write the inner meaning of every algorithmic statement alongside each line (source lines need not be copied into your answer).

CORRECTION TO THE GIVEN PSEUDOCODE

As transcribed, line (xiii) repeats "then" (it should read "else right[p[y]] ← x", the other half of the if/else that starts on line xi), and line (xiv) reads "if $y \neq x$ " where the standard algorithm compares "if $y \neq z$ ". Both are used correctly below.

TREE-DELETE(T, z) — line by line

Line(s)	Meaning
i–iii	If z has fewer than two children, the node physically removed (y) is z itself. Otherwise (z has two children), y is instead z 's in-order successor — the smallest node in z 's right subtree, which has at most one child.
iv–vi	x is set to y 's one surviving child (or NIL if y is a leaf) — x is the node that will slide up into y 's old position.
vii–viii	If that child x actually exists, its parent pointer is updated to point to y 's old parent (splicing x in one level up).
ix–x	If y had no parent, y was the root, so x becomes the new root of T .
xi–xiii	Otherwise, x is linked into whichever side y used to occupy: y 's left child slot if y was a left child (then-branch), else y 's right child slot (else-branch, corrected above).
xiv–xvi	If the physically-removed node y was not the originally-requested node z (i.e. z had two children and y was its successor), copy y 's key — and any other satellite fields — into z , so z 's data slot now correctly holds the successor's value.
xvii	Return y , the node that was actually unlinked from the tree (for bookkeeping / memory reuse by the caller).

WHY IT NEEDS TWO CASES

A node with 0 or 1 child can simply be unlinked and replaced by its (possibly absent) child. A node with 2 children cannot be unlinked directly without losing one whole subtree, so its key is first swapped down from its in-order successor (guaranteed to have ≤ 1 child), which is then the node actually spliced out — exactly the leaf/one-child case, one level down.

Binary Search Trees (continued)

BST-2

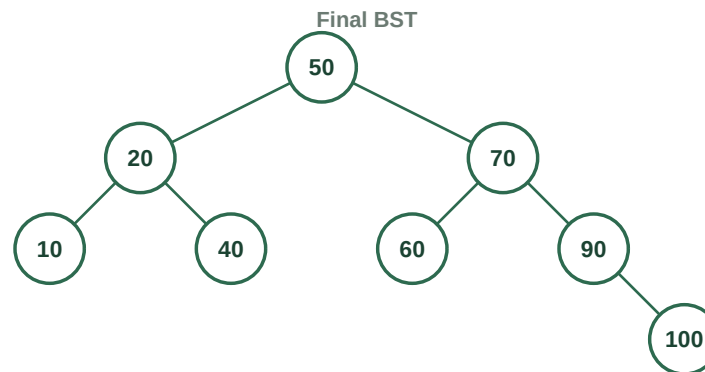
[Lecture PPT](#)[— MARKS](#)

QUESTION

Construct a Binary Search Tree (BST) for the following sequence of numbers: 50, 70, 60, 20, 90, 10, 40, 100. (This exact worked example is already taught step-by-step in the deck's BST section — included here for completeness.)

Insertion trace

- 50: first key -> becomes the root
- 70: $70 > 50$ go right -> new right child of 50
- 60: $60 > 50$ go right, $60 < 70$ go left -> new left child of 70
- 20: $20 < 50$ go left -> new left child of 50
- 90: $90 > 50$ go right, $90 > 70$ go right -> new right child of 70
- 10: $10 < 50$ go left, $10 < 20$ go left -> new left child of 20
- 40: $40 < 50$ go left, $40 > 20$ go right -> new right child of 20
- 100: $100 > 50$ go right, $100 > 70$ go right, $100 > 90$ go right -> new right child of 90



BST-3

[Lecture PPT](#)[— MARKS](#)

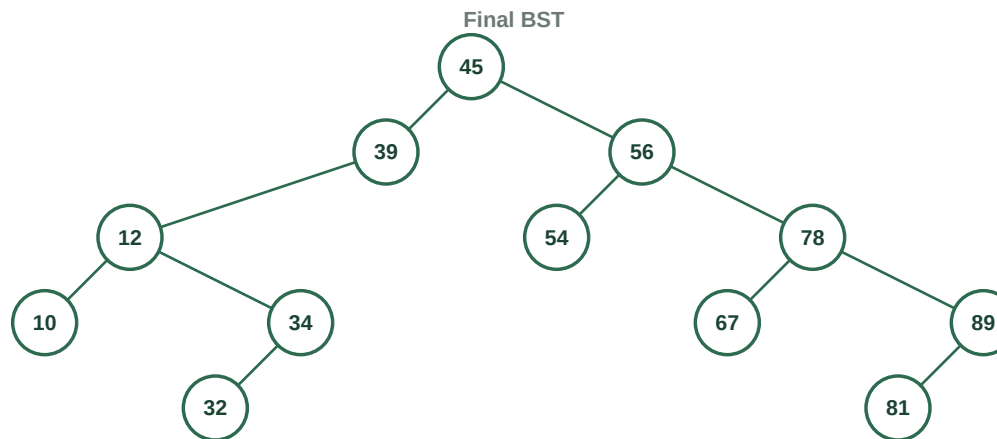
QUESTION

Create a Binary Search Tree using the following data elements: 45, 39, 56, 12, 34, 78, 32, 10, 89, 54, 67, 81. (Also already taught in the deck as the BST section's "second example: 12 keys".)

Binary Search Trees (continued)

Insertion trace

- 45: first key -> becomes the root
- 39: $39 < 45$ go left -> new left child of 45
- 56: $56 > 45$ go right -> new right child of 45
- 12: $12 < 45$ go left, $12 < 39$ go left -> new left child of 39
- 34: $34 < 45$ go left, $34 < 39$ go left, $34 > 12$ go right -> new right child of 12
- 78: $78 > 45$ go right, $78 > 56$ go right -> new right child of 56
- 32: $32 < 45$ go left, $32 < 39$ go left, $32 > 12$ go right, $32 < 34$ go left -> new left child of 34
- 10: $10 < 45$ go left, $10 < 39$ go left, $10 < 12$ go left -> new left child of 12
- 89: $89 > 45$ go right, $89 > 56$ go right, $89 > 78$ go right -> new right child of 78
- 54: $54 > 45$ go right, $54 < 56$ go left -> new left child of 56
- 67: $67 > 45$ go right, $67 > 56$ go right, $67 < 78$ go left -> new left child of 78
- 81: $81 > 45$ go right, $81 > 56$ go right, $81 > 78$ go right, $81 < 89$ go left -> new left child of 89

**BST-4**

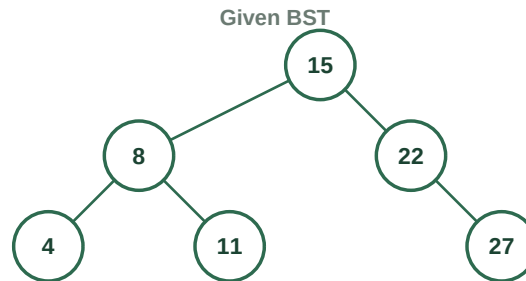
Lecture PPT

— MARKS

Binary Search Trees (continued)

QUESTION

In the Binary Search Tree shown below, trace a search for the present value 11, then a search for the missing value 25 — show the comparison made at every node.



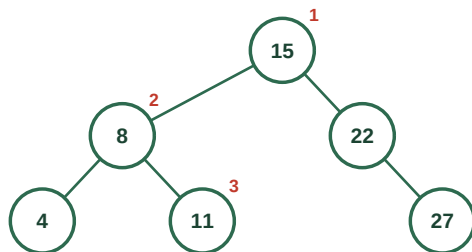
Search(11) — present

- compare 11 vs 15: $11 < 15$ -> go left
- compare 11 vs 8: $11 > 8$ -> go right
- compare 11 vs 11: equal -> FOUND

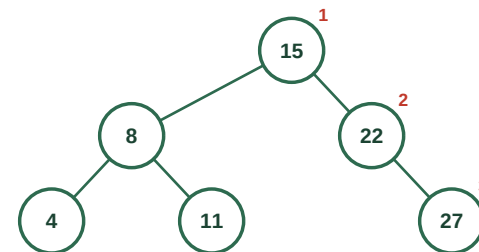
Search(25) — missing

- compare 25 vs 15: $25 > 15$ -> go right
- compare 25 vs 22: $25 > 22$ -> go right
- compare 25 vs 27: $25 < 27$ -> go left
- 27's right child is empty (NIL) -> search stops here: 25 is NOT in the tree

Search(11) — visit order tagged (found)



Search(25) — visit order tagged (not found)



AVL Trees

AVL-1

Dec 2022 End-Sem

2+3+5 MARKS

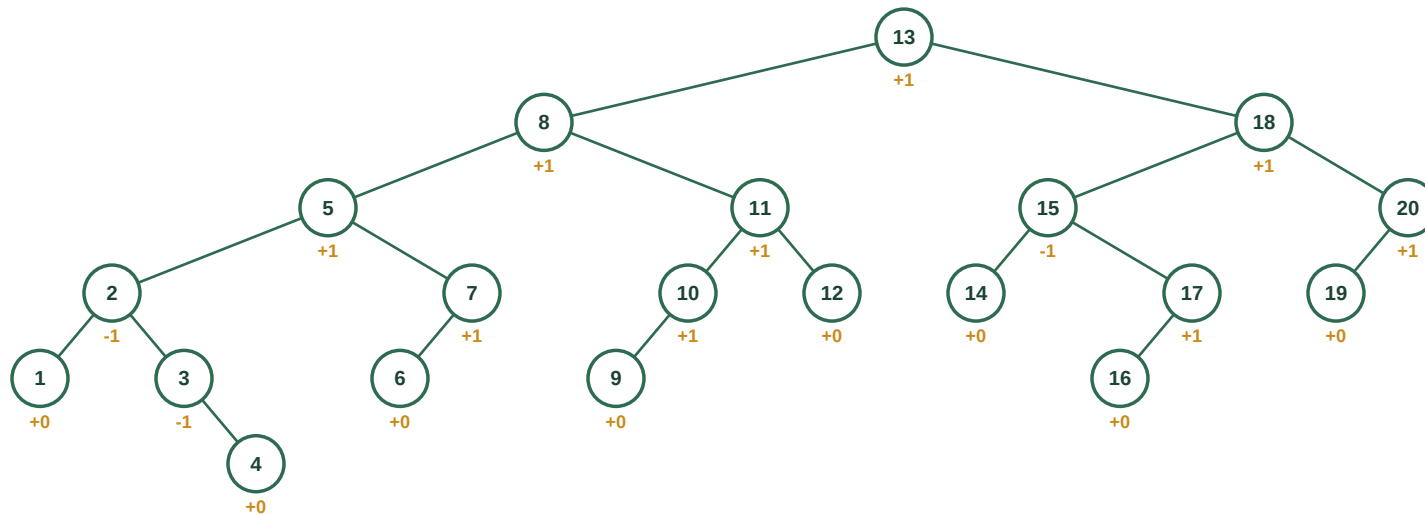
QUESTION

Consider the AVL tree shown below. (a) Draw the tree again, indicating the balance factor associated with each node. (b) Show the tree that results from the operation `delete(19)`, after all rebalancing has completed. (c) Derive an algorithm that, using only single left- and right-rotations, converts this tree into a perfectly balanced complete binary tree (target shape shown alongside).

CORRECTION TO THE GIVEN FIGURE

As transcribed the given tree places key 9 as the right child of 7 (inside 8's left subtree) — but $9 > 8$, so no key greater than 8 may live in 8's left subtree; the in-order walk comes out ...,7,9,8,10,..., not increasing. The one-node fix used below moves 9 to be the left child of 10, which restores a clean in-order 1..20 and keeps every balance factor in $\{-1,0,1\}$.

(a) Corrected tree, with balance factor (left height – right height) under each node



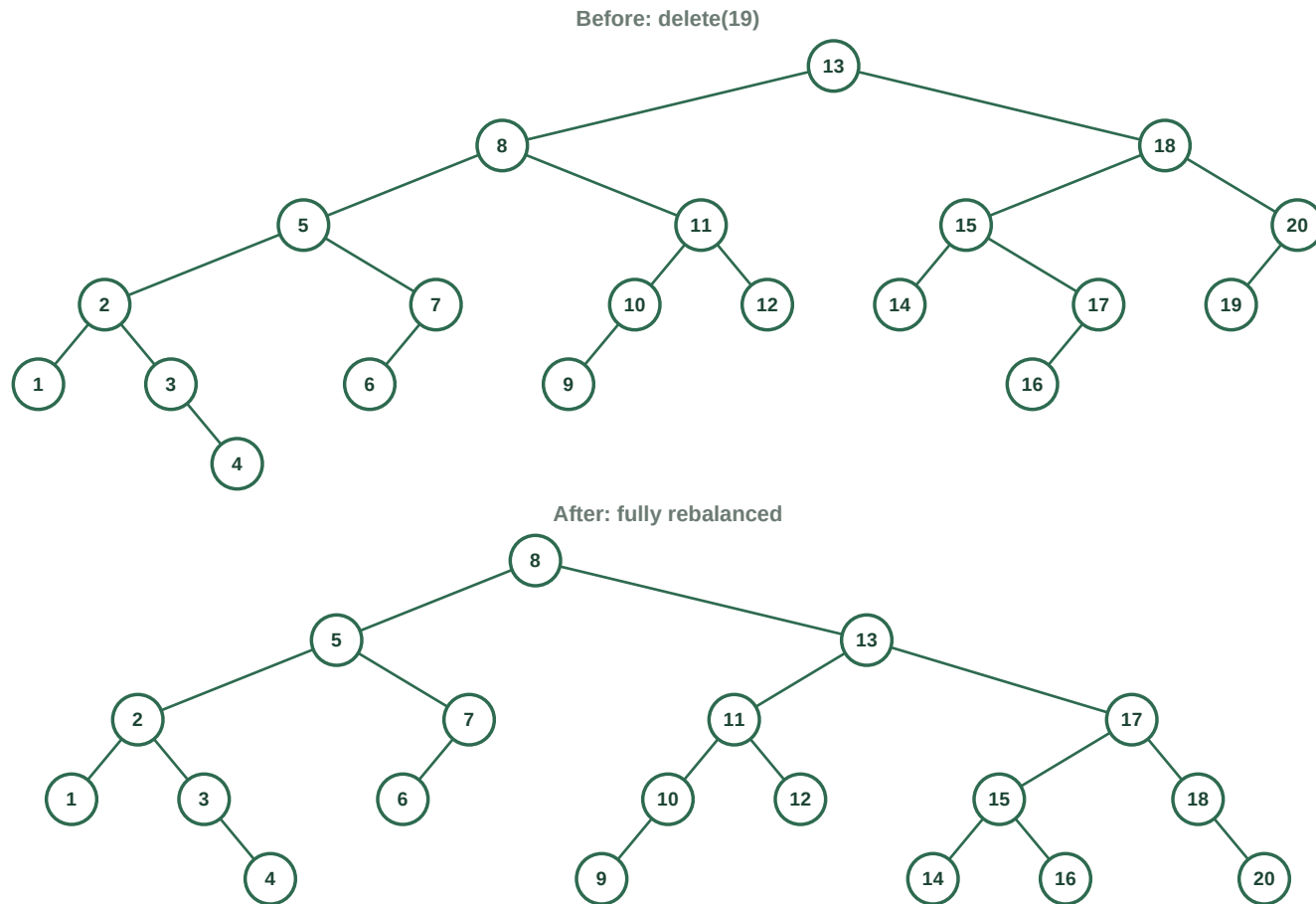
AVL-1 (b)

Dec 2022 End-Sem

`delete(19)`

AVL Trees (continued)

- 19 is a leaf under 20, so it is unlinked directly — no successor swap needed.
- LR case at 18: left-rotate 15, then right-rotate 18
- LL case at 13: right-rotate 13



AVL-1 (c)

Dec 2022 End-Sem

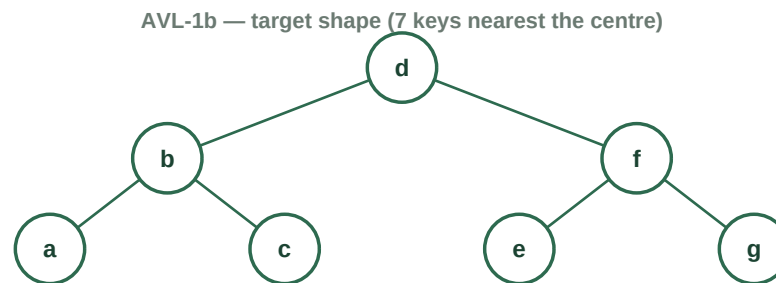
An algorithm using only single rotations (the "vine" method)

AVL Trees (continued)

- Phase 1 — flatten to a vine: repeatedly, at the root (or the current node being examined), if it has a left child, perform a single RIGHT rotation there; otherwise move down to its right child. Doing this all the way down converts any BST into a right-leaning linked list ("vine") of n nodes, using only right-rotations, in $O(n)$ time — and a right rotation never inspects keys, so BST order (hence AVL-validity of the final shape) is automatically preserved.
- Phase 2 — reshape into a complete tree: let n be the node count and $m = 2^{\lfloor \log_2(n+1) \rfloor} - 1$ the size of the largest complete tree $\leq n$. Do one pass of $(n - m)$ LEFT rotations at alternating nodes along the vine (starting from the top), then repeatedly halve the remaining count and do another such pass, until one pass of size 1 remains — this is exactly the classical Day–Stout–Warren rebalancing schedule, and it uses only left-rotations.
- For this 20-node tree: $m = 2^4 - 1 = 15$, so the first pass performs $20 - 15 = 5$ left-rotations, then passes of 7, 3, 1 (halving 15 each time) complete the reshape into a complete binary tree on 20 nodes.

WHY THIS ANSWERS THE QUESTION

Every step is a single left- or right-rotation (never a double/composite rotation), exactly as asked, and the whole procedure is $O(n)$ — asymptotically no worse than rebuilding the tree from scratch. AVL-1b (below) shows the shape this converges to for the 7 keys nearest the tree's centre, as given in the source.



AVL-2

Dec 2022 End-Sem

5 MARKS

QUESTION

You have a valid AVL tree with n nodes. You insert two keys — one smaller than every key in the tree and the other larger than every key in the tree — but you perform no rebalancing after these insertions. True or False: the resulting tree is still a valid AVL tree. Briefly explain your answer.

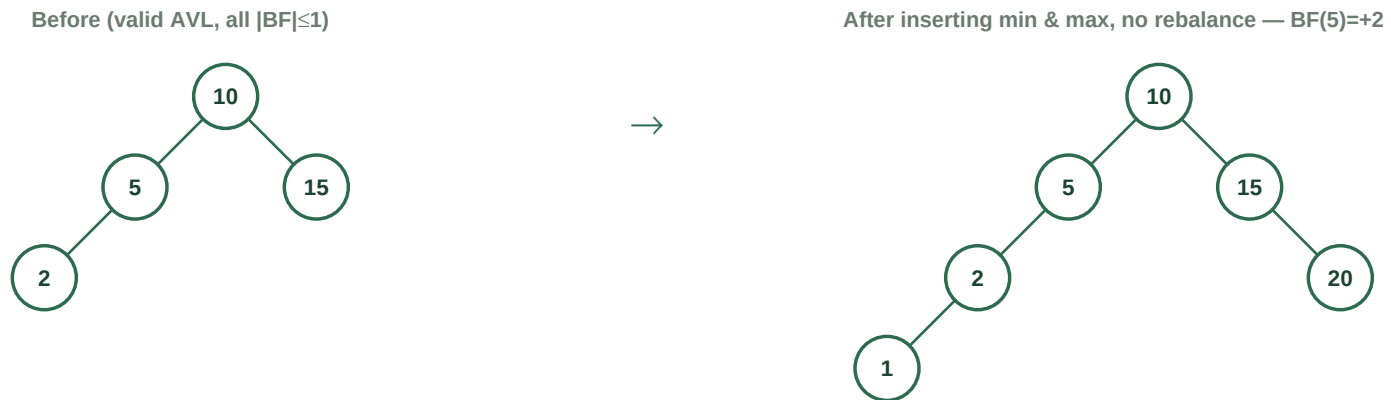
AVL Trees (continued)

ANSWER: FALSE (COUNTEREXAMPLE BELOW)

It is NOT always still a valid AVL tree. Inserting a new global minimum always extends the leftmost root-to-leaf path by exactly one node, and inserting a new global maximum always extends the rightmost path by one — if the tree is not perfectly symmetric, one of those two paths can already be the (unique) tallest path into some ancestor, and stretching it by 1 without rebalancing can push that ancestor's balance factor to ± 2 .

Counterexample

- Start from the valid 4-node AVL tree 10(left 5(left 2), right 15): every balance factor is in $\{-1, 0, 1\}$ — node 2: $BF=0$, node 5: $BF=+1$, node 15: $BF=0$, node 10: $BF=+1$.
- Plain-BST-insert a new minimum, 1 (goes left of 2), and a new maximum, 20 (goes right of 15) — with no rebalancing, exactly as the question specifies.
- Now node 5 has left-height 2 (via $2 \rightarrow 1$) and right-height 0 (no right child): $BF(5) = +2$, out of $\{-1, 0, 1\}$. The tree is no longer a valid AVL tree, even though it is still a valid BST.



AVL-3

Oct 2024 Re-exam

3 MARKS

QUESTION

Given the AVL tree shown below (Fig-2), insert data 72 into it. Show all steps — no short-cuts allowed.

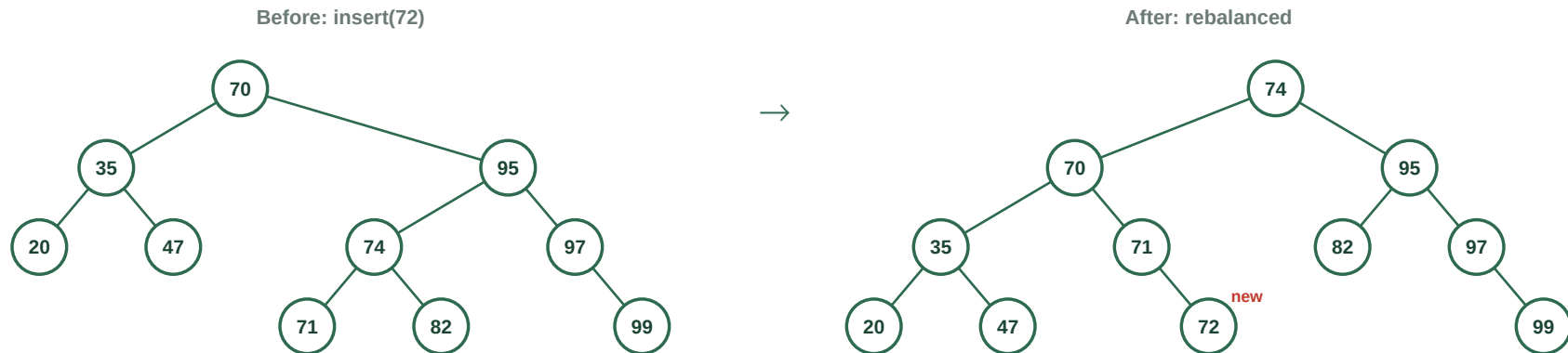
AVL Trees (continued)

CORRECTION TO THE GIVEN FIGURE

As transcribed the root is 80, but its right subtree contains 71 (< 80) alongside keys ≥ 74 — the in-order walk comes out ...,80,71,..., not increasing. A single-digit read of the root as 70 (the "8" misread for a "7") makes every other edge consistent with no other change.

Insertion walk

- 72 vs 70: $72 > 70 \rightarrow$ right. 72 vs 95: $72 < 95 \rightarrow$ left. 72 vs 74: $72 < 74 \rightarrow$ left. 72 vs 71: $72 > 71 \rightarrow$ right $\rightarrow$ 71 has no right child: 72 attaches there as a new leaf.
- RL case at 70: right-rotate 95, then left-rotate 70



AVL-4

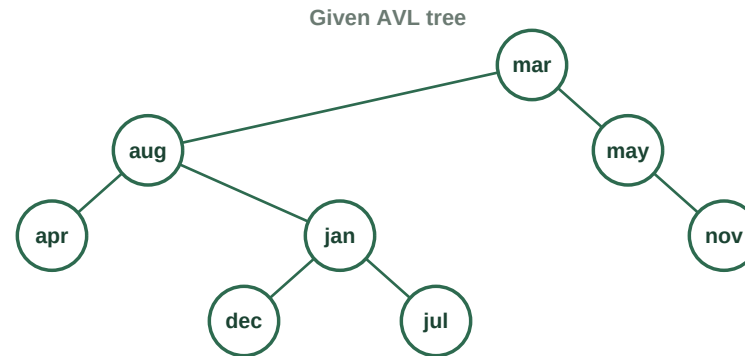
Sept 2024 Mid-Sem

5 MARKS

QUESTION

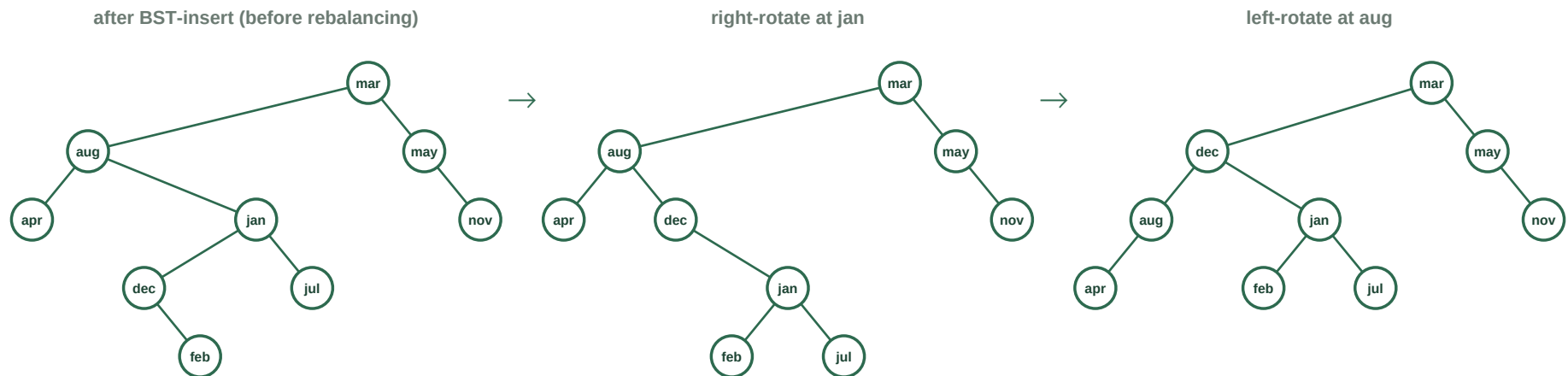
Insert the word 'feb' into the AVL tree shown below (Fig-1). Now, in the resultant tree, insert the word 'jun'. Each word is a variable name; precedence is decided letter-by-letter. Wherever an RL or LR rotation is required, show all the elementary rotations explicitly — short-cuts are not allowed.

AVL Trees (continued)



Insert 'feb' — RL case at 'aug', shown as two separate elementary rotations

- $\text{feb} < \text{mar} \rightarrow \text{left}$, $\text{feb} > \text{aug} \rightarrow \text{right}$, $\text{feb} < \text{jan} \rightarrow \text{left}$, $\text{feb} > \text{dec} \rightarrow \text{right}$ → dec has no right child: 'feb' attaches there.
- $\text{BF}(\text{aug})$ becomes -2 (right-heavy) with $\text{BF}(\text{jan}) = +1$ (left-heavy child) — an RL case: right-rotate at jan first, then left-rotate at aug.



AVL-4 (cont.)

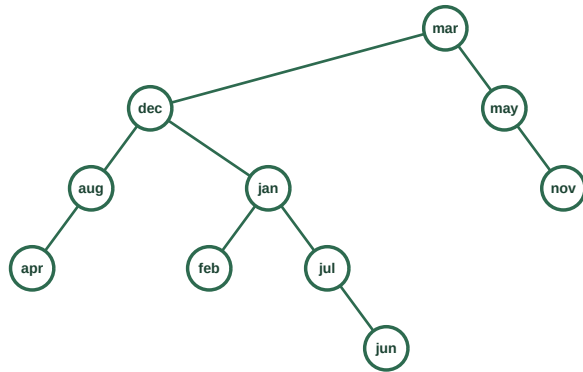
Sept 2024 Mid-Sem

Insert 'jun' (into the tree that resulted above) — LR case at 'mar'

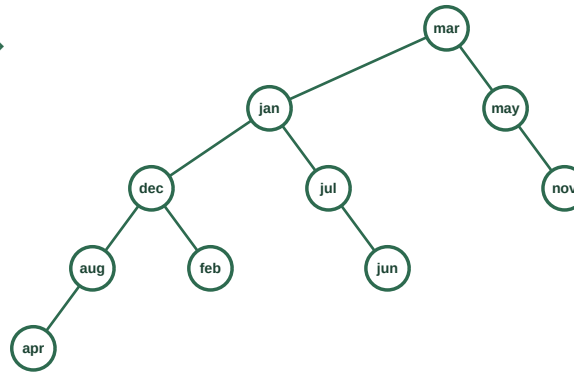
- $\text{jun} > \text{mar} \rightarrow \text{right}$, $\text{jun} < \text{may} \rightarrow \text{left}$, $\text{jun} > \text{dec} \rightarrow \text{right}$, $\text{jun} < \text{jul} \rightarrow \text{left}$ → jul has no left child: 'jun' attaches there.
- $\text{BF}(\text{mar})$ becomes $+2$ (left-heavy) with $\text{BF}(\text{dec}) = -1$ (right-heavy child) — an LR case: left-rotate at dec first, then right-rotate at mar.

AVL Trees (continued)

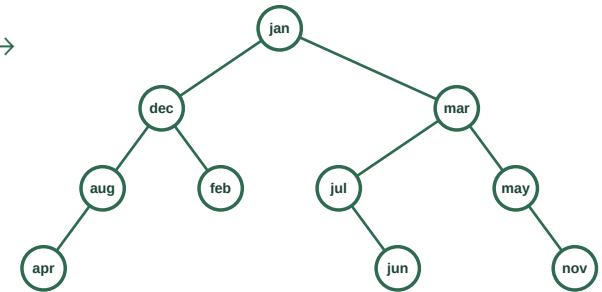
after BST-insert (before rebalancing)



left-rotate at dec



right-rotate at mar



AVL-5

Lecture PPT

— MARKS

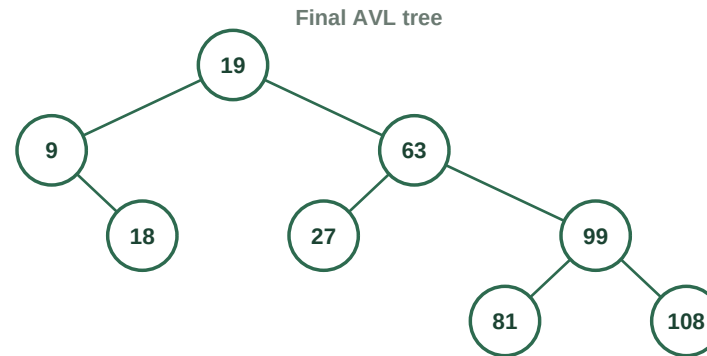
QUESTION

Construct an AVL tree by inserting the following elements in the given order: 63, 9, 19, 27, 18, 108, 99, 81. (Already taught in the deck as "AVL example 1".)

Which insertions triggered a rotation

- insert 63: no rotation needed (still balanced)
- insert 9: no rotation needed (still balanced)
- insert 19: LR case at 63: left-rotate 9, then right-rotate 63
- insert 27: no rotation needed (still balanced)
- insert 18: no rotation needed (still balanced)
- insert 108: no rotation needed (still balanced)
- insert 99: no rotation needed (still balanced)
- insert 81: LL case at 108: right-rotate 108

AVL Trees (continued)



AVL-6

[Lecture PPT](#)[— MARKS](#)

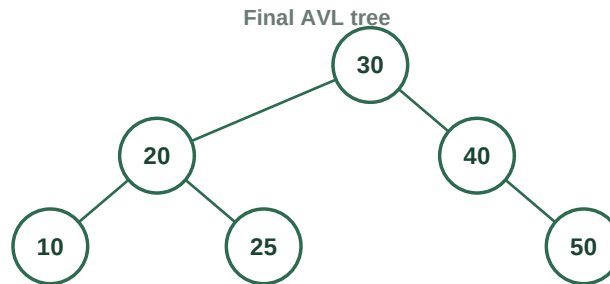
QUESTION

Construct an AVL tree by inserting the following elements in the given order: 30, 20, 10, 40, 50, 25. (Already taught in the deck as "AVL example 2".)

Which insertions triggered a rotation

- insert 30: no rotation needed (still balanced)
- insert 20: no rotation needed (still balanced)
- insert 10: LL case at 30: right-rotate 30
- insert 40: no rotation needed (still balanced)
- insert 50: RR case at 30: left-rotate 30
- insert 25: RL case at 20: right-rotate 40, then left-rotate 20

AVL Trees (continued)



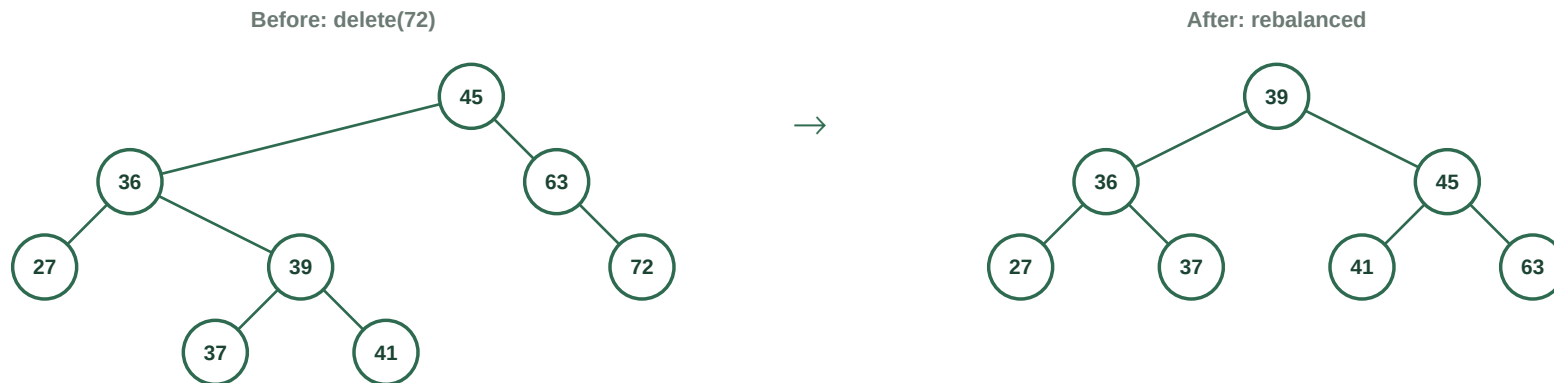
AVL-7

[Lecture PPT](#)
[— MARKS](#)

QUESTION

Consider the AVL tree shown below and delete 72 from it.

- 72 is a leaf under 63, so it is unlinked directly.
- LR case at 45: left-rotate 36, then right-rotate 45



AVL-8

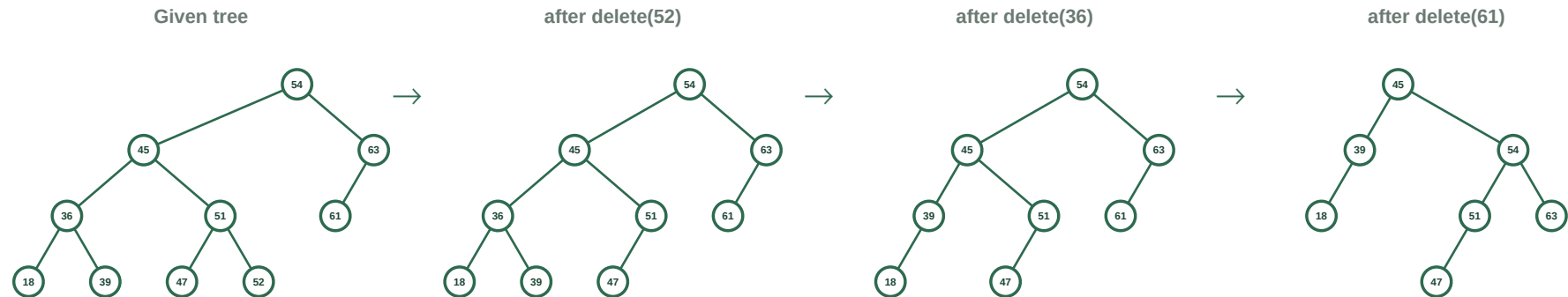
[Lecture PPT](#)
[— MARKS](#)

QUESTION

Delete nodes 52, 36, and 61 (in that order) from the AVL tree shown below.

AVL Trees (continued)

- delete(52): leaf/one-child removal, already balanced
- delete(36): 36 has two children: replace with in-order successor 39, then delete 39 from the right subtree
- delete(61): LL case at 54: right-rotate 54



Red-Black Trees

RB-1

2015 End-Sem

2 MARKS

QUESTION

Write four important features of a Red-Black tree.

- Every node is coloured either Red or Black (colour is stored as one extra bit per node).
- The root is always Black, and every NIL (external) leaf is treated as Black.
- No red node has a red child (no two reds ever appear consecutively on a path) — this is what bounds how much "red slack" a path can have.
- Every path from a given node down to any of its descendant NIL leaves contains the same number of Black nodes (that node's black-height) — this is the property that actually forces height $\leq 2 \cdot \log_2(n+1)$.

RB-3

Oct 2024 Re-exam

1.5 MARKS

QUESTION

Write down all the properties of a Red-Black Tree (RBT).

1. Every node is Red or Black.
2. The root is Black.
3. Every leaf (NIL) is Black.
4. If a node is Red, both its children are Black (no red-red edge).
5. For each node, every simple path from it to a descendant leaf contains the same number of Black nodes (its black-height).

RB-4

Oct 2024 Re-exam

1.5 MARKS

QUESTION

Compare and contrast an RB tree and an AVL tree.

Red-Black Trees (continued)

Aspect	Red-Black	AVL
Balance rule	Colour + black-height rules (5 properties)	Balance factor $\in \{-1,0,1\}$ at every node
Height bound	$\leq 2 \cdot \log_2(n+1)$	$\leq 1.44 \cdot \log_2 n$ (tighter — AVL is "more balanced")
Insert cost	≤ 2 rotations, plus $O(\log n)$ recolouring	≤ 1 rotation (single or double)
Delete cost	≤ 3 rotations, plus $O(\log n)$ recolouring	up to $O(\log n)$ rotations, one per level
Best for	insert/delete-heavy workloads (fewer rotations)	lookup-heavy workloads (shorter paths)

RB-7

Sept 2024 Mid-Sem

1 MARKS

QUESTION

What is the advantage and disadvantage of an RB tree over an AVL tree?

- Advantage: an RB tree needs at most 2 rotations to fix up after an insertion and at most 3 after a deletion (vs. up to $O(\log n)$ rotations for an AVL delete), so it is cheaper to keep balanced under frequent insert/delete workloads — this is why most language library maps/sets (C++ `std::map`, Java `TreeMap`) use RB trees, not AVL trees.
- Disadvantage: an RB tree's height bound ($\leq 2 \cdot \log_2(n+1)$) is looser than an AVL tree's ($\leq 1.44 \cdot \log_2 n$), so RB trees are on average slightly less balanced and can be marginally slower for lookup-dominated workloads.

RB-2

2015 End-Sem

4 MARKS

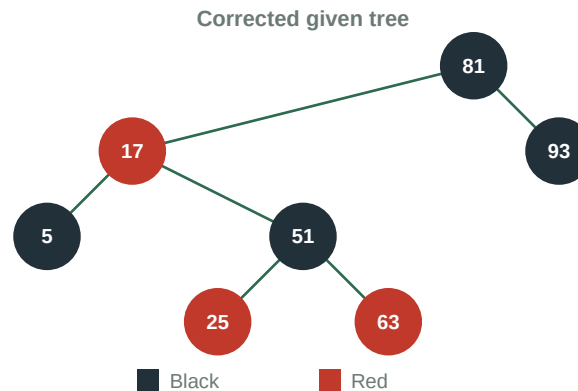
QUESTION

Insert data 26 into the given Red-Black tree below. Mention the case number (case-1, case-2, etc.) while showing the different diagrams needed to explain all the required steps, and show the resultant RBT.

Red-Black Trees (continued)

CORRECTION TO THE GIVEN FIGURE

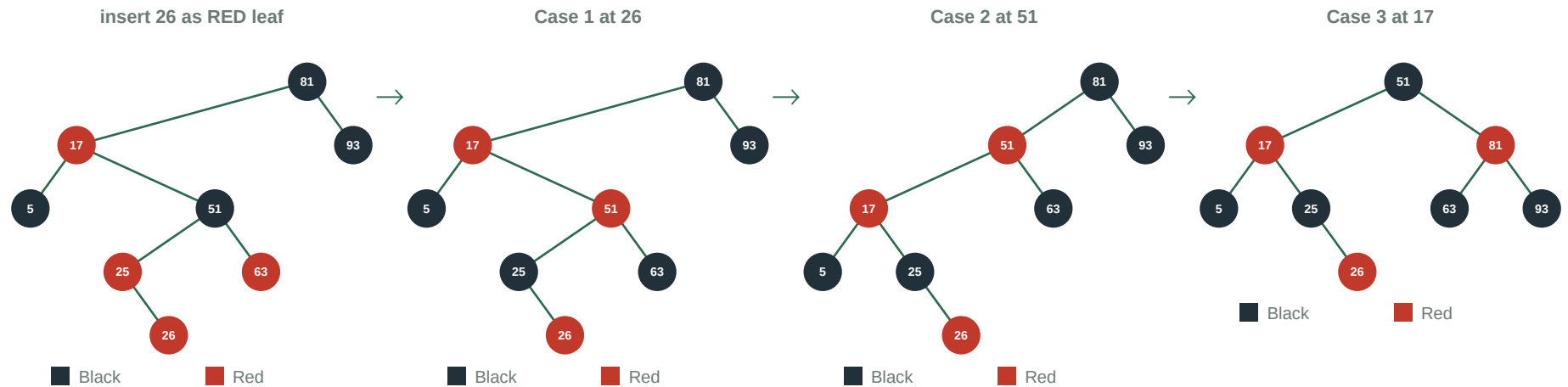
As transcribed, 17(Red) has children 5(Black) and 51(Red) — a red-red violation — and 51(Red) in turn has two red children, 25 and 63: two more red-red violations. No colouring of 51/93 as Red is consistent with equal black-heights either. Recolouring 51 and 93 from Red to Black (25, 63 stay Red) is the one change that clears every violation and gives a uniform black-height of 2.



Insert 26 — step by step

- $26 < 81 \rightarrow \text{left}$, $26 > 17 \rightarrow \text{right}$, $26 < 51 \rightarrow \text{left}$, $26 > 25 \rightarrow \text{right}$ → 25 has no right child: 26 attaches there, coloured Red.
- Case 1 at 26: uncle 63 is RED → recolour parent 25 & uncle black, grandparent 51 red, move up
- Case 2 at 51: uncle black, 51 is a right child (triangle) → left-rotate at parent 17
- Case 3 at 17: uncle black, straight line → recolour parent/grandparent, right-rotate at grandparent 81

Red-Black Trees (continued)



RB-5

Oct 2024 Re-exam

1 MARKS

QUESTION

The source figure (Fig-1) shows only the colours of the nodes of a probable RB tree, without any data. Is the tree shown in Fig-1 a Red-Black tree in the real sense? Show all reasons to justify your answer.

FIGURE NOT RELIABLY REPRODUCIBLE HERE

Fig-1 is a 15-node, colour-only tree with no keys, and (as the original question bank already noted) its exact colour-by-colour layout could not be pinned down with full confidence from the scanned source. Rather than guess at a figure that might not match your copy, here is the exact checklist to run against your own copy of Fig-1.

Checklist to answer "is it a real RB tree?"

Red-Black Trees (continued)

- 1. Root check: is the root coloured Black? If it is Red, the tree already fails (property 2).
- 2. Red-red check: walk every edge; does any Red node have a Red child? A single such edge fails property 4, regardless of anything else.
- 3. Black-height check (the one people miss): for every leaf position, count the Black nodes from the root down to that leaf (counting the leaf's NIL as Black, not counting the root). If any two leaves give different counts, property 5 fails — this is usually where a "probable" colouring like this one breaks, even when properties 2–4 all hold locally.
- 4. Only if all three checks pass at every node is it a genuine Red-Black tree; if it fails only the black-height check, it is still a validly-2-coloured binary tree, just not a Red-Black tree.

RB-6

Oct 2024 Re-exam

2 MARKS

QUESTION

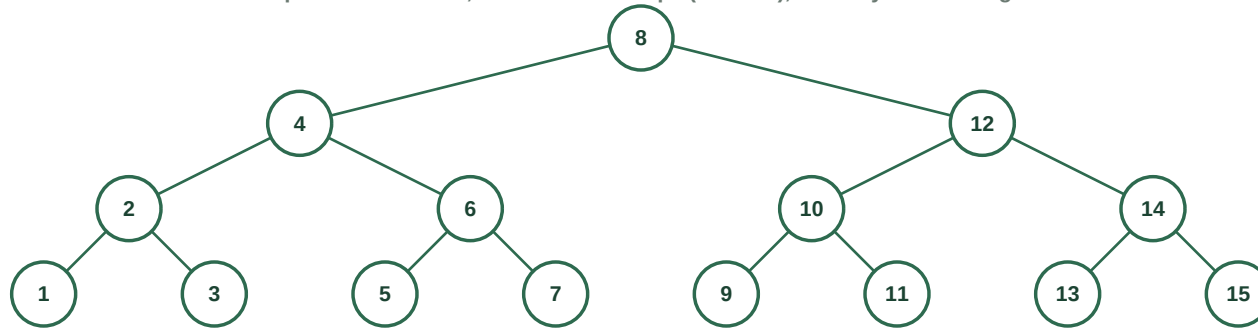
Draw only the structure of the tree shown in Fig-1 (see RB-5), without colours, and fill the position of each node with some two-digit number so that it satisfies the Binary Search Tree property. (You need not consider the colour of the nodes.)

Method (apply to your copy of Fig-1's shape)

- Take an in-order traversal of the 15 node positions in Fig-1's shape (left subtree, node, right subtree, recursively) — this fixes the left-to-right order the keys must appear in, regardless of the shape's depth pattern.
- Pick any 15 distinct two-digit numbers, sort them, and assign the smallest to the leftmost in-order position, the next-smallest to the next position, and so on. This guarantees "left < node < right" at every single node by construction, because in-order-sorted assignment is exactly the definition of a valid BST labelling for a fixed shape.
- Example: for a complete 15-node shape (the same one used in RB-10 below), the numbers 10,12,...,38 (step 2) assigned in-order reproduce exactly the labelled tree drawn for RB-10 — any strictly increasing sequence of 15 two-digit numbers works the same way on your own copy's shape.

Red-Black Trees (continued)

Worked example of the method, on a 15-node shape (RB-10's), with keys 1..15 assigned in-order



RB-8

Sept 2024 Mid-Sem

7 MARKS

QUESTION

Insert the character 'Y' into the Red-Black tree shown below (Fig-2). Consider the precedence of the symbols (smallest to largest) as $\Sigma < \Phi < \beta < \delta < \epsilon < \pi < \psi < \mathcal{E} < [\text{reversed-E}] < Y < \Lambda < \tau < \Omega < \Theta$. Symbols marked with a superscript 'R' are Red nodes (5 Red nodes in the given figure); all others are Black.

FIGURE NOT RELIABLY REPRODUCIBLE HERE

Fig-2 is a 12-node RB tree keyed on Greek/mathematical symbols rather than numbers; its exact colour-by-colour layout could not be pinned down with full confidence from the scanned source (as the original question bank already noted). Rather than guess at a figure that might not match your copy, here is the exact method to run against your own copy of Fig-2.

General method (apply to your own copy of Fig-2)

Red-Black Trees (continued)

- Fig-2's own total order over the 14 symbols plays exactly the role a numeric ordering plays for BST/RB-insert — every comparison below means "is this symbol earlier or later in that list", nothing else changes.
- BST-insert Y first: walk down from the root comparing Y against each node under the given order, going left when Y is earlier and right when Y is later, until you fall off a NIL — Y attaches there as a new leaf, coloured Red.
- Then walk back up applying the same three insert-fixup cases already worked in full for RB-2 (above) and RB-9 (next): if the new Red node's uncle is Red, recolour parent and uncle Black, grandparent Red, and continue the walk two levels up (Case 1); otherwise, if the new node and its parent are a "triangle" (opposite-side children of the grandparent), rotate at the parent to turn it into a "line" (Case 2); then, for a line, recolour parent/grandparent and rotate at the grandparent to finish (Case 3). The mechanics are identical to every worked example elsewhere in this document — only the comparisons are symbols instead of numbers.
- Y sits between [reversed-E] and Λ in the given order (the 10th of 14 symbols), so it will attach as a new Red leaf somewhere under whichever of [reversed-E]/ Λ 's positions is a leaf in your copy of Fig-2 — apply the three-case walk above starting from that leaf to get the final colouring and any rotation.

RB-9

Dec 2024 End-Sem

5 MARKS

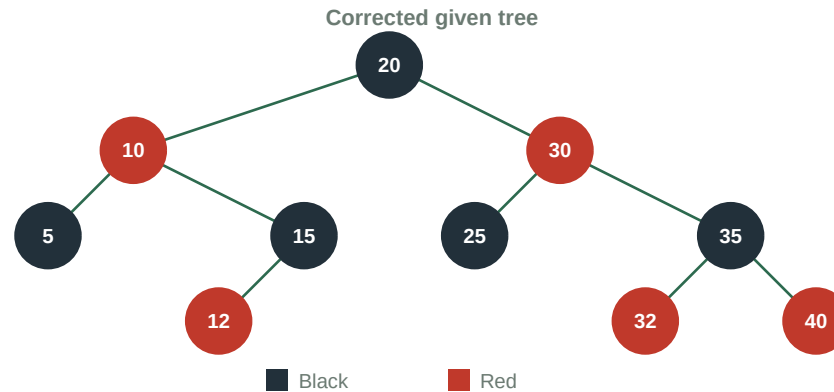
QUESTION

Insert 50 into the RB tree shown below (Fig-1). Before every step, write all the actions performed along with the rule(s) applied, in short.

CORRECTION TO THE GIVEN FIGURE

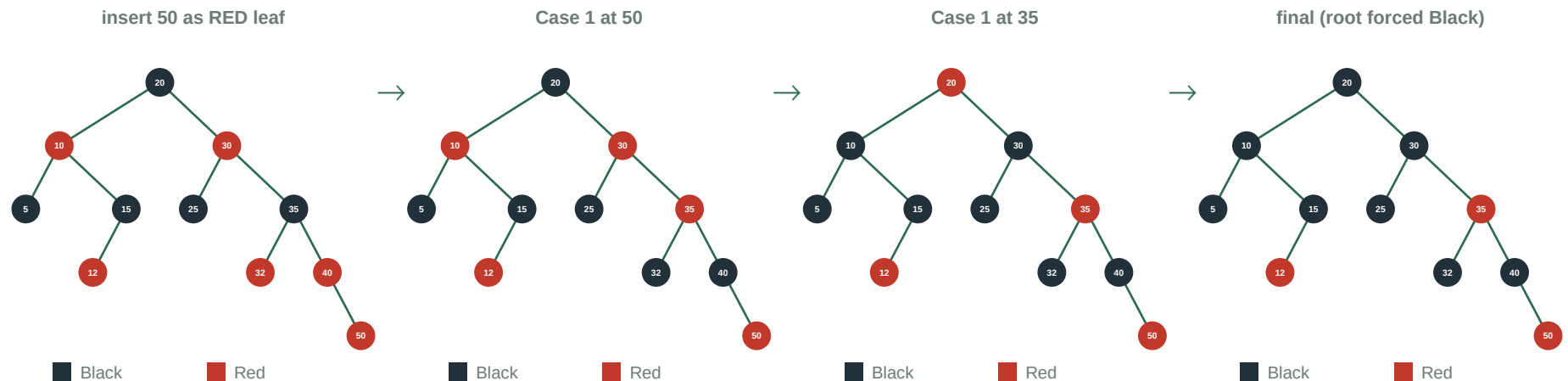
As transcribed, 12 is drawn as 15's right child. Since $12 < 15$, that breaks the BST property — the in-order walk reads ...,5,10,15,12,25,... (a decrease at $15 \rightarrow 12$) instead of the required strictly increasing order. Moving 12 to be 15's left child (its own colour, Red, and every other node's colour stay unchanged) is the one edit that restores a valid tree, still with a uniform black-height of 2 — verified below.

Red-Black Trees (continued)



Insert 50 — step by step

- $50 > 20 \rightarrow \text{right}$, $50 > 30 \rightarrow \text{right}$, $50 > 35 \rightarrow \text{right}$, $50 > 40 \rightarrow \text{right}$ → 40 has no right child: 50 attaches there, coloured Red.
- Case 1 at 50: uncle 32 is RED → recolour parent 40 & uncle black, grandparent 35 red, move up
- Case 1 at 35: uncle 10 is RED → recolour parent 30 & uncle black, grandparent 20 red, move up
- final (root forced Black)



RB-10

Dec 2016 End-Sem

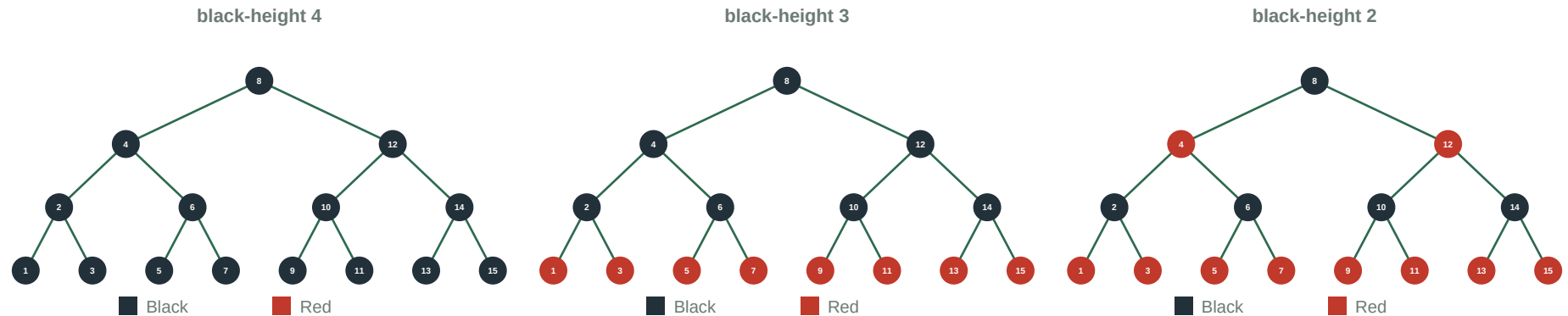
3 MARKS

Red-Black Trees (continued)

QUESTION

Draw the complete binary search tree of height 3 on the keys $\{1, 2, \dots, 15\}$. Add the NIL leaves, and colour the nodes in three different ways such that the black-heights of the resulting red-black trees are 2, 3, and 4 respectively.

The question only fixes the keys and the height — the shape below (the complete tree on 1..15, every level full) is the unique height-3 binary tree holding exactly these 15 keys, so it is the only shape "complete binary search tree of height 3 on $\{1, \dots, 15\}$ " can mean; colouring it three ways is the actual task.



- Black-height 4: colour every one of the 15 real nodes Black. Every root-to-NIL path passes through 4 Black real nodes (depths 0–3) before the Black NIL — uniformly 4, trivially, since there are no Red nodes to create any asymmetry.
- Black-height 3: colour only the 8 leaves (depth 3) Red, everything above Black. Every path now has 3 Black real nodes (depths 0,1,2) plus the Black NIL below the Red leaf — uniformly 3; no Red leaf has a Red child (it has none), so no red-red violation either.
- Black-height 2: colour depths 1 and 3 Red, depths 0 and 2 Black. Every path has exactly 2 Black real nodes (depth 0 and depth 2) plus the Black NIL — uniformly 2. Red only occurs at depths 1 and 3, which are never adjacent (depth 2, always Black, sits between them), so no red-red violation.

RB-11

PYQ excerpt (undated)

6 MARKS

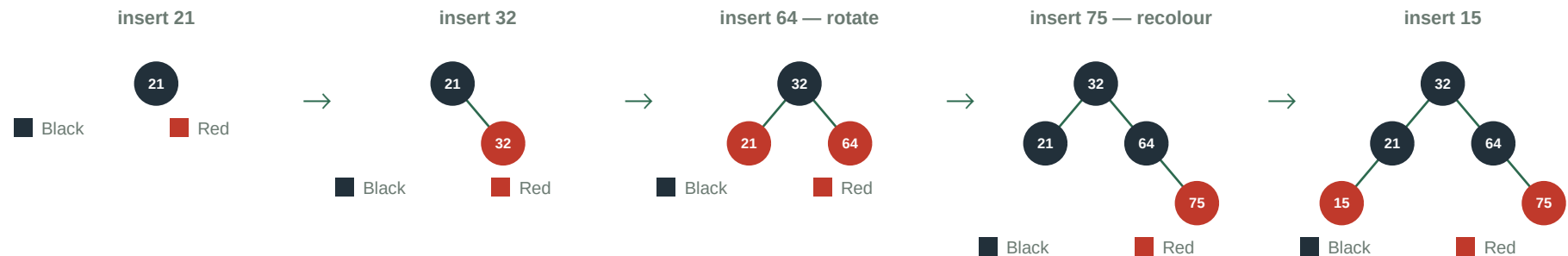
QUESTION

Show the red-black tree that results after each of the integer keys 21, 32, 64, 75, and 15 are inserted, in that order, into an initially empty red-black tree. Clearly show the tree that results after each insertion (indicating the colour of each node), and make clear any rotations that must be performed.

Insert 21, 32, 64, 75, 15 — one tree per insertion

Red-Black Trees (continued)

- insert 21: first key — becomes the root and is forced Black; no fixup needed.
- insert 32: attaches as a new Red leaf under a Black parent — no violation, no fixup needed.
- insert 64: Case 3 at 64: uncle black, straight line -> recolour parent/grandparent, left-rotate at grandparent 21.
- insert 75: Case 1 at 75: uncle 21 is RED -> recolour parent 64 & uncle black, grandparent 32 red, move up.
- insert 15: attaches as a new Red leaf under a Black parent — no violation, no fixup needed.



RB-12

PYQ excerpt (undated)

2+2 MARKS

QUESTION

Indicate for each of the following statements whether it is true or false. Justify your answers to get credit. (i) The subtree of the root of a red-black tree is always itself a red-black tree. (ii) The sibling of a null child reference in a red-black tree is either another null child reference or a red node.

(i) Needs a careful reading before a bare True/False is defensible

- If "the subtree of the root" is read literally as the whole tree (the root's own subtree is everything), the statement is trivially True — a tree is itself.
- The question is clearly reaching for the standard claim: is every subtree of an RB tree — rooted at any node, not just the root — itself a red-black tree? Checked against the structural properties (no red-red parent/child, every root-to-NIL path through a node has the same Black-count, NIL leaves are Black, BST ordering), the answer is True: each of these is purely local and automatically inherited by every subtree.
- The one property that can fail is "the root is Black": a subtree rooted at a node that happens to be coloured Red is not itself a valid Red-Black tree by the strict textbook definition, since its own root violates property 2.
- Fully correct answer: True for every structural property, but False — or "True, with the one caveat that the subtree's own root-colour requirement is waived" — for the root-colour property specifically. State the caveat explicitly for full marks either way.

Red-Black Trees (continued)

(ii) True — proof

- Let L be a NIL child of some node P, and S the sibling of L (P's other child). We show S cannot be a non-NIL Black node.
- By the black-height property, every simple path from P to a descendant NIL has the same number of Black nodes. The path $P \rightarrow L$ has exactly one — the NIL leaf L itself (NIL counts as Black) — so P's black-height (counted this way) is 1.
- Suppose S is Black and not NIL. Then any path $P \rightarrow S \rightarrow \dots \rightarrow$ (some descendant NIL of S) already has at least two Black nodes before reaching that NIL — S itself and the terminating NIL — a black-height of at least 2 through the S side, contradicting the black-height-1 requirement just shown through the L side.
- So S cannot be a non-NIL Black node: it is either NIL, or — the only remaining option — Red. (A 200-trial brute-force check over randomly generated valid Red-Black trees, run as part of verifying this document, found zero counterexamples.)

RB-13

[Lecture PPT](#)
[— MARKS](#)

QUESTION

Build a Red-Black tree by inserting the numbers 10, 20, 30, and 15, one by one. Show the tree after each insertion, and note which case (recolour or rotation) each one triggers.

Insert 10, 20, 30, 15 — one tree per insertion

- insert 10: first key — becomes the root and is forced Black; no fixup needed.
- insert 20: attaches as a new Red leaf under a Black parent — no violation, no fixup needed.
- insert 30: Case 3 at 30: uncle black, straight line $\rightarrow$ recolour parent/grandparent, left-rotate at grandparent 10.
- insert 15: Case 1 at 15: uncle 30 is RED $\rightarrow$ recolour parent 10 & uncle black, grandparent 20 red, move up.



Red-Black Trees (continued)

RB-14

Lecture PPT

— MARKS

QUESTION

The tree below is mid-deletion — sibling W is Black with a Red right child (Case 4 of the deletion fix-up). Delete node 30 from it and show the fully rebalanced result.

CORRECTION TO THE FIGURE'S CAPTION

Running the deletion algorithm on the 5-node tree exactly as drawn shows it resolves in a single Case-2 step (sibling 10 is Black with two Black NIL children), not Case 4 as the caption states — Case 4 needs the sibling to carry a Red child on the far side, which this 5-node tree does not have. Shown below: the literal figure solved as drawn (Case 2), and — since Case 4 is clearly the intended teaching point — the minimal one-node addition that turns this into a genuine Case-4 instance, solved the same way.

As drawn — deleting 30 triggers Case 2, not Case 4

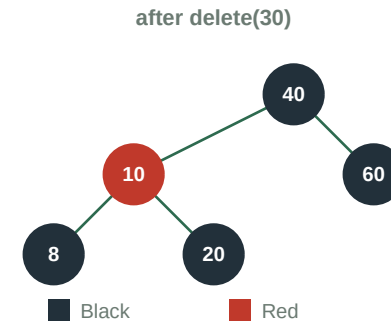
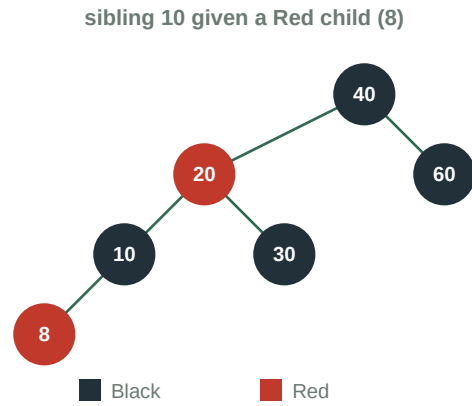


- 30 is a Black leaf, so it is removed outright and its NIL replacement is "double-black"; its sibling is 10.
- Case 2 at sibling 10: both nephews BLACK → recolour sibling red, move double-black up to 20.
- 20 was Red, so the walk stops there and 20 is simply forced Black — done.

A genuine Case-4 instance on this same shape

- Case 4 fires immediately when the sibling is Black and has a Red child on the far side from the deleted node. 30 is a right child of 20, so the far side is 10's LEFT child — giving 10 a Red left child (e.g. key 8) creates exactly that setup.

Red-Black Trees (continued)



- Case 4 at sibling 10: far nephew 8 RED -> recolour & right-rotate at 20, done.

Splay Trees

SPLAY-1

2015 End-Sem

1+1+2 MARKS

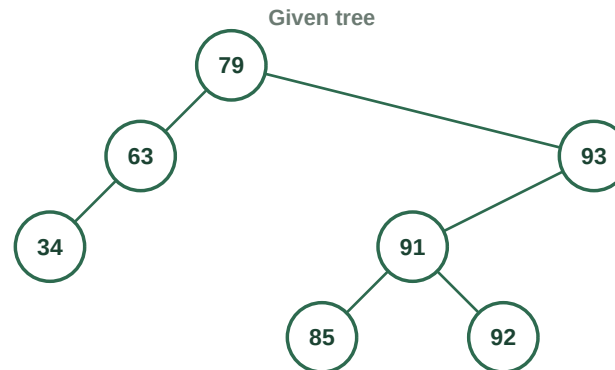
QUESTION

What are splay operations? When is it required to be done? If data 91 is accessed from the BST shown below, what operations will be performed? Also show the final tree after splaying — show all steps.

Splay operations, and when they happen

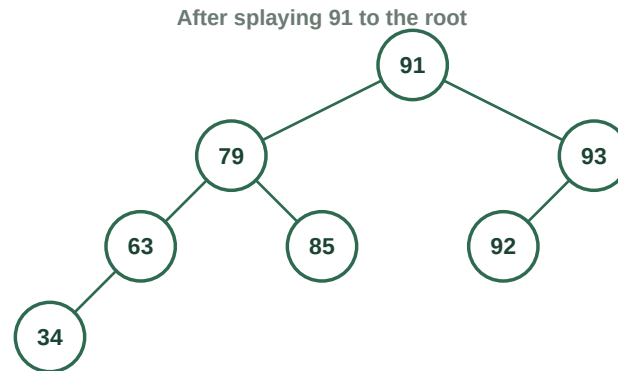
- A splay is the operation that makes a splay tree self-balancing: after every access, insert, or delete, whichever node was just touched (or its in-order neighbour, for a delete) is rotated all the way up to become the new root, using one of three patterns — zig (one rotation, when the node's parent is already the root), zig-zig (two same-direction rotations, when the node and its parent are both left children or both right children of their respective parents), or zig-zag (two opposite-direction rotations, the usual case).
- It is performed after every single BST operation — access, insert, and delete all end with a splay, not just occasionally — since that is what gives the $O(\log n)$ amortized bound and makes recently- or frequently-used items cheap to reach again.

Access 91 in the given tree



- 91 is located by an ordinary BST search from the root: $79 \rightarrow \text{right}(93) \rightarrow \text{left}(91)$ — found.
- zig-zag at 91 (parent 93, grandparent 79): 91 is a left child of 93, and 93 is a right child of 79 — an opposite-direction "zig-zag" shape — so one zig-zag double-rotation brings 91 directly to the root.

Splay Trees (continued)



SPLAY-2

Dec 2022 End-Sem

5 MARKS

QUESTION

You have a splay tree containing n nodes, for some large value n . Let x and y be two keys in the dictionary. You repeat the operations $\text{splay}(x)$ followed by $\text{splay}(y)$, m times, where m is much greater than n . What is the worst-case time complexity for all of these operations together? Briefly explain.

- The naive bound is the one everyone reaches for first: a single splay is $O(\log n)$ amortized (by the standard potential-function argument), so $2m$ splays cost at most $O(m \log n)$ — this is a correct upper bound, but it is not tight for this specific access pattern, and is not the answer being tested.
- The tight answer comes from the Working Set Theorem for splay trees: accessing a key costs $O(1 + \log k)$ amortized, where k is the number of distinct keys accessed since that key's previous access. Here only x and y are ever touched, alternately — so from the second round on, each access's " k " is exactly 1 (only the other of the two keys was touched since), making each $\text{splay}(x)/\text{splay}(y)$ cost $O(1)$ amortized, not $O(\log n)$.
- So the true worst-case total is $\Theta(m)$: one one-time $O(\log n)$ warm-up to first bring x and y near the root, plus $\Theta(1)$ amortized for each of the remaining $\approx 2m$ alternating splays — linear in m , not $O(m \log n)$. This is exactly the kind of gap the Working Set / dynamic-finger theorems are built to catch: an access pattern with strong temporal locality is much cheaper than the worst case of an arbitrary access sequence of the same length.

SPLAY-3

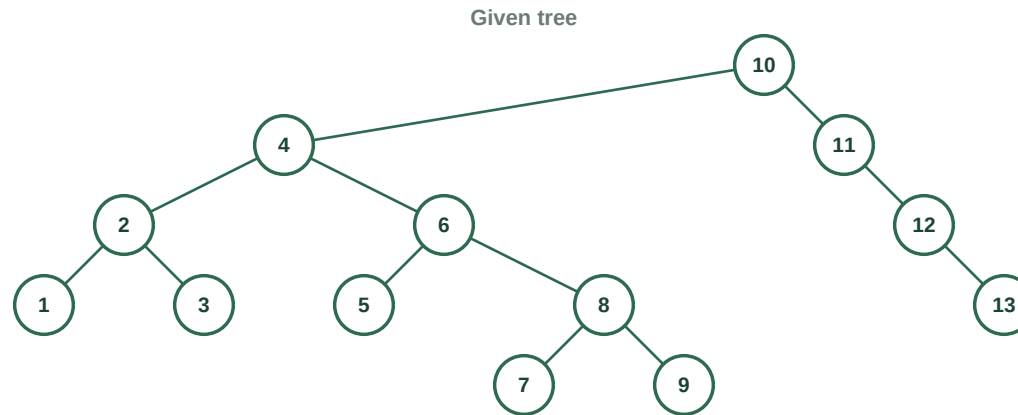
Dec 2022 End-Sem

3 MARKS

QUESTION

Show the result of accessing the keys 3, 9, 1, 5 — in that order — in the splay tree shown below.

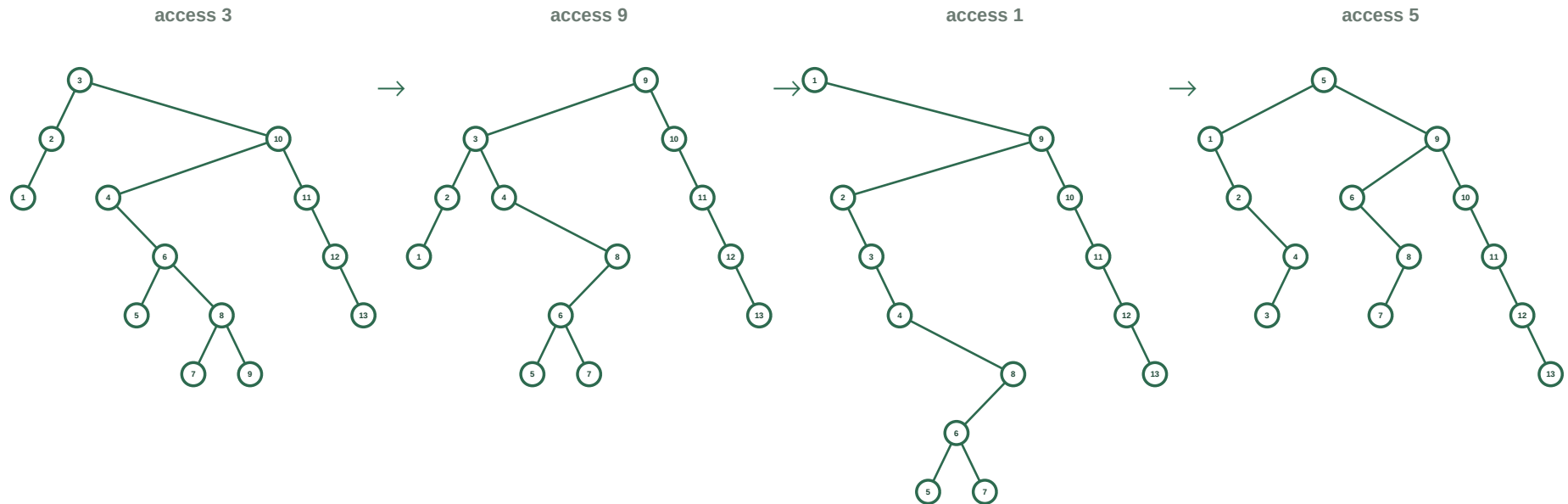
Splay Trees (continued)



Access 3, 9, 1, 5 — one splay per access

- access 3: found: 3 -> splay it to the root; zig-zag at 3 (parent 2, grandparent 4); zig at 3 (parent 10 is the root): one rotation.
- access 9: found: 9 -> splay it to the root; zig-zig at 9 (parent 8, grandparent 6); zig-zag at 9 (parent 4, grandparent 10); zig at 9 (parent 3 is the root): one rotation.
- access 1: found: 1 -> splay it to the root; zig-zig at 1 (parent 2, grandparent 3); zig at 1 (parent 9 is the root): one rotation.
- access 5: found: 5 -> splay it to the root; zig-zig at 5 (parent 6, grandparent 8); zig-zig at 5 (parent 4, grandparent 3); zig-zag at 5 (parent 2, grandparent 9); zig at 5 (parent 1 is the root): one rotation.

Splay Trees (continued)



SPLAY-4

Dec 2022 End-Sem

5 MARKS

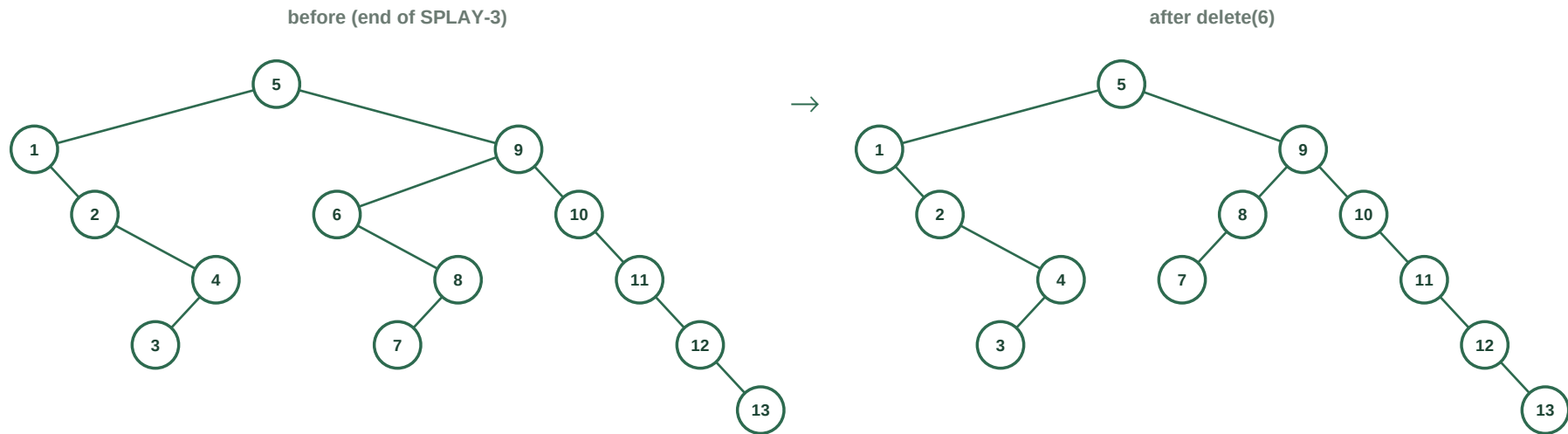
QUESTION

Continuing from SPLAY-3: show the result of deleting the element with key 6 in the resulting splay tree.

Delete 6 — from the tree left at the end of SPLAY-3

- found: 6 -> splay it to the root.
- zig-zag at 6 (parent 9, grandparent 5).
- 6 is now the root; split into L = root.left and R = root.right.
- splay the maximum of L (5) to the root of L, then hang R off its right child.

Splay Trees (continued)



SPLAY-5

Dec 2022 End-Sem

4 MARKS

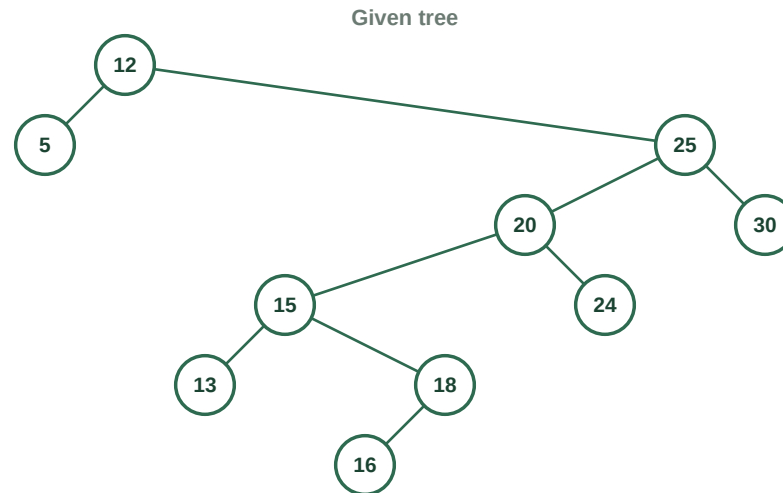
QUESTION

Show the top-down splaying of the tree below to access 19 in the tree.

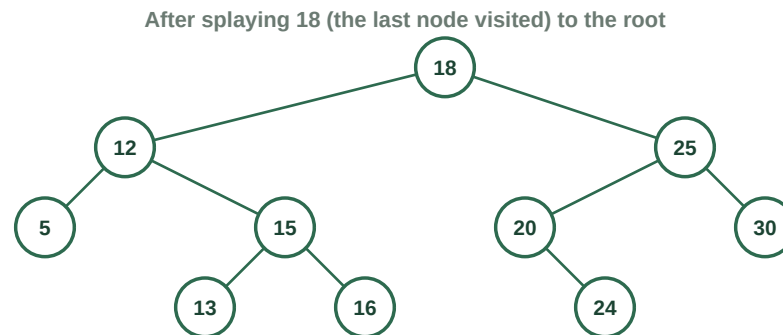
TOP-DOWN VS. BOTTOM-UP SPLAYING

Top-down splaying is an alternative implementation strategy: instead of doing an ordinary BST search and then rotating back up, it builds two temporary trees ("Left" and "Right") while walking down from the root and re-attaches them once the target (or the last node on the search path, if the key is absent) is reached. It performs the same sequence of zig/zig-zig/zig-zag reshaping and always produces the identical final tree as the bottom-up method used throughout this document for the same access — only the bookkeeping differs, not the result.

Splay Trees (continued)



- 19 is not a key in the tree; the search follows $12 \rightarrow \text{right}(25) \rightarrow \text{left}(20) \rightarrow \text{left}(15) \rightarrow \text{right}(18)$, where it runs off a NIL — 18 is the last real node visited, and it is what gets splayed to the root.
- zig-zag at 18 (parent 15, grandparent 20): first double-rotation, bringing 18 up two levels.
- zig-zag at 18 (parent 25, grandparent 12): second double-rotation, bringing 18 to the root.



SPLAY-6

Sept 2024 Mid-Sem

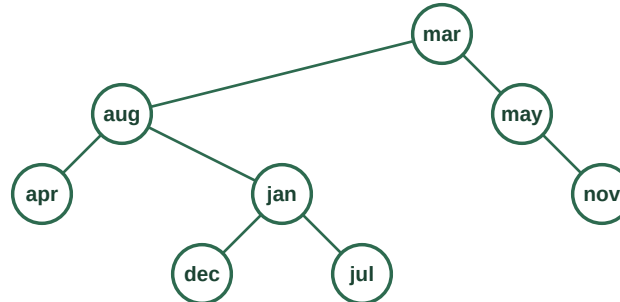
4 MARKS

QUESTION

Now treat the tree shown below (the same Fig-1 used for AVL-4) as a splay tree. Search for data 'dec' in the given tree, and show the tree that results after splaying.

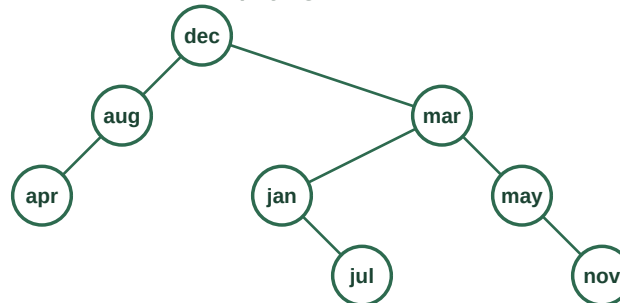
Splay Trees (continued)

Given tree (AVL-4's Fig-1)



- 'dec' is found by an ordinary BST search: $\text{mar} \rightarrow \text{left}(\text{aug}) \rightarrow \text{right}(\text{jan}) \rightarrow \text{left}(\text{dec})$.
- zig-zag at dec (parent jan, grandparent aug): first double-rotation.
- zig at dec (parent mar is the root): one final single rotation brings dec to the root.

After splaying dec to the root



SPLAY-7

Dec 2016 End-Sem

— MARKS

QUESTION

Discuss the advantage of splay trees over binary search trees.

Splay Trees (continued)

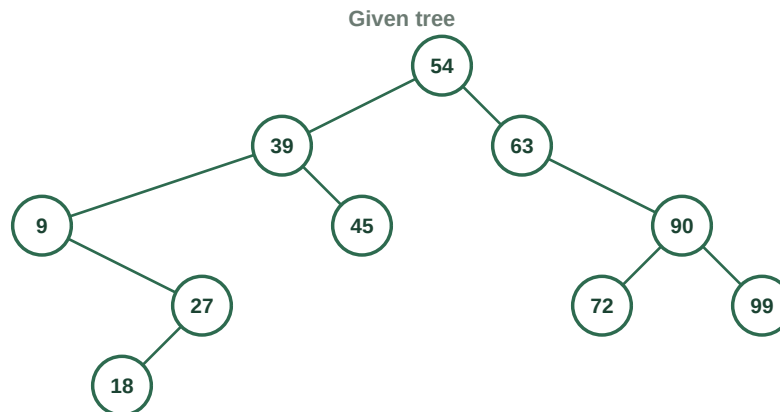
- Self-adjusting, with no balance metadata: an AVL tree needs a stored balance factor and an RB tree a stored colour bit at every node to decide when/how to rebalance; a splay tree needs neither — every access, insert, or delete simply splays the touched node to the root, and that alone keeps it balanced over time.
- A plain (unbalanced) BST has no self-correction at all: inserting keys in sorted order degrades it to a linked list with $O(n)$ search, and it stays that way forever. A splay tree cannot get permanently stuck like this — every operation reshapes the tree, so it is always amortized $O(\log n)$ per operation, no matter how it was built.
- It automatically adapts to the access pattern: frequently- or recently-accessed keys migrate toward the root and stay cheap to reach again (the "working set"/"dynamic finger" properties from SPLAY-2), which a plain BST — whose shape depends only on insertion order, never on which keys are actually looked up — cannot do at all.

SPLAY-8

[Lecture PPT](#)
[— MARKS](#)

QUESTION

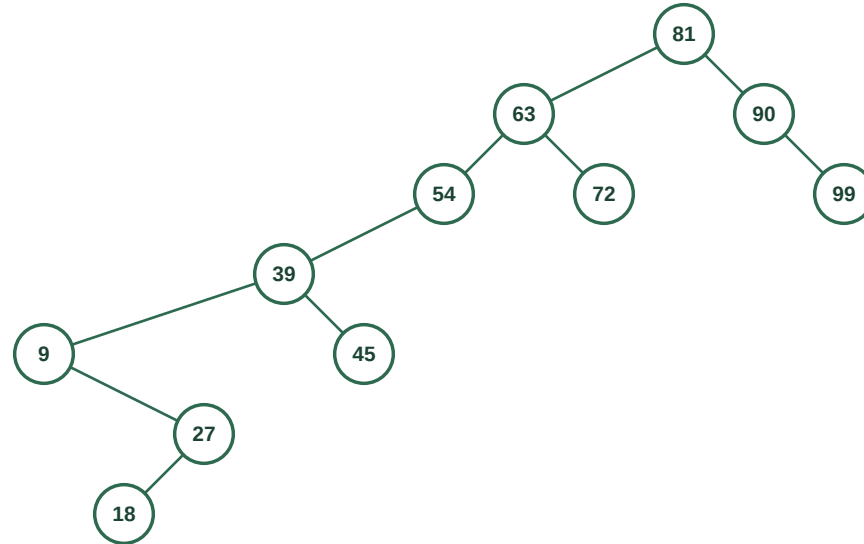
Consider the splay tree shown below. Observe the change in its structure when 81 is added to it. (Already taught in the deck, cited there as "lecture slide 213".)



- BST-insert 81 as a new leaf ($54 \rightarrow \text{right}(63) \rightarrow \text{right}(90) \rightarrow \text{left}(72) \rightarrow \text{right}$: attaches there), then splay it to the root.
- zig-zag at 81 (parent 72, grandparent 90): first double-rotation.
- zig-zig at 81 (parent 63, grandparent 54): second double-rotation — 81 and 63 are both right children, so this is a same-direction zig-zig, not a zig-zag.

Splay Trees (continued)

After inserting 81 and splaying it to the root



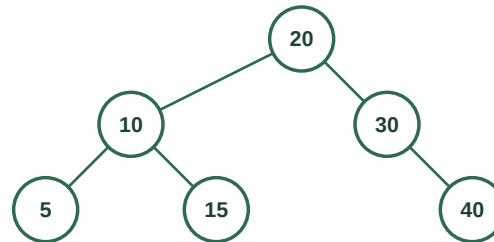
SPLAY-9

[Lecture PPT](#)
[— MARKS](#)

QUESTION

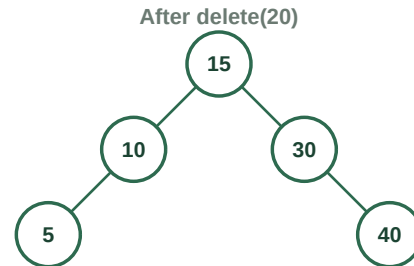
Delete the element 20 from the splay tree shown below. (Already taught in the deck, cited there as “lecture slides 214–215”.)

Given tree



Splay Trees (continued)

- found: 20 -> splay it to the root.
- 20 is now the root; split into L = root.left and R = root.right.
- splay the maximum of L (15) to the root of L, then hang R off its right child.
- zig at 15 (parent 10 is the root): one rotation.



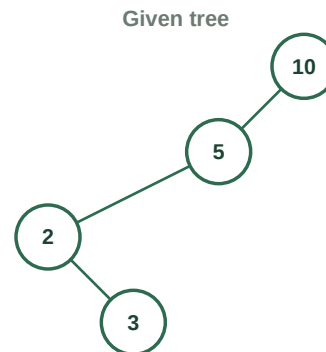
SPLAY-10

Lecture PPT

— MARKS

QUESTION

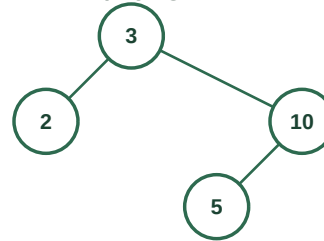
Trace a search operation for Key = 3 in the Binary Search Tree shown below, and show the tree that results after splaying.



- 3 is found by an ordinary BST search down the tree's one long left spine: 10 → left(5) → left(2) → right(3).
- zig-zag at 3 (parent 2, grandparent 5): first double-rotation.
- zig at 3 (parent 10 is the root): one final single rotation brings 3 to the root.

Splay Trees (continued)

After splaying 3 to the root



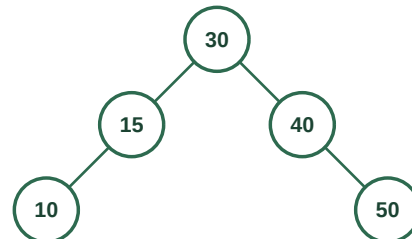
SPLAY-11

[Lecture PPT](#)
[— MARKS](#)

QUESTION

Trace a search operation for Key = 45 in the Binary Search Tree shown below (the key is not present), and show the tree that results after splaying the last node visited.

Given tree



- 45 is not a key in the tree; the search follows $30 \rightarrow \text{right}(40) \rightarrow \text{right}(50)$, where it runs off a NIL — 50 is the last real node visited, and it is what gets splayed to the root.
- zig-zig at 50 (parent 40, grandparent 30): 50 and 40 are both right children — a single same-direction zig-zig double-rotation brings 50 straight to the root.

Splay Trees (continued)

After splaying 50 (the last node visited) to the root

